

Jenkins Master Slave Architecture

Class no:4

Master-Agent architecture

In Jenkins, the **Master-Agent architecture** (historically and still commonly referred to as the **Master-Slave architecture**) is a design pattern used to distribute the workload of building, testing, and deploying software.

Instead of forcing a single server to handle every single automation task, Jenkins offloads the actual work to a fleet of separate machines.

The Core Components

1. The Jenkins Controller (The "Master")

Think of the Controller as the **brain** or the project manager. It doesn't (or shouldn't) do the heavy lifting itself. Instead, its primary responsibilities include:

- Hosting the main user interface (UI) and dashboard.
- Scheduling build jobs based on triggers (like a Git commit or a timer).
- Parsing and configuration management for pipelines.
- Dispatching build steps to the available agents.
- Monitoring agents and recording their build results, logs, and artifacts.

2. The Jenkins Agents (The "Slaves")

Think of Agents as the **muscle** or the workers. These are separate machines (physical servers, virtual machines, Docker containers, or cloud instances) that run a small Java executable listener to communicate with the controller.

- They wait for instructions from the controller.
- They execute the actual build steps (e.g., compiling code, running tests, building Docker images).
- They send the execution logs and status back to the controller.

Why Use This Architecture?

Using a distributed setup offers several massive benefits over running everything on a single server:

- **Scalability:** If you have dozens of development teams running hundreds of builds a day, a single machine will quickly run out of CPU and RAM. You can infinitely scale by just adding more agents.
- **Environment Isolation:** Different projects require different environments. For example, you might need an iOS build to run on a macOS agent, a .NET app to build on a Windows agent, and a Python app to build on a Linux agent.
- **Security:** You can restrict sensitive production deployment permissions to specific, hardened agents, keeping the main controller isolated from executing untrusted code.
- **High Availability:** If an agent crashes mid-build, only that specific job fails. The Jenkins controller remains up, and other jobs keep running smoothly on other agents.

How They Communicate

The controller and agents usually connect in one of two ways:

1. **SSH (Inbound to Agent):** The controller initiates a secure connection to the agent machine using SSH credentials. This is the standard for Linux-based agents.
2. **JNLP / Inbound (Outbound from Agent):** The agent machine initiates a TCP connection *to* the controller. This is highly useful when the agent is behind a firewall or running on a Windows machine.

Hands on: Creating a master slave architecture via AWS EC2

Step no:1 Create a slave instance with Ubuntu machine

Name and tags [Info](#)

Name

Slave-Machine

[Add additional tags](#)

▼ Application and OS Images (Amazon Machine Image) [Info](#)

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use [Browse more AMIs](#).

Q Search our full catalog including 1000s of application and OS images

RecentsQuick Start

Amazon Linux

aws

macOS

Mac

Ubuntu

Canonical Ubuntu

Windows

Microsoft

Red Hat

Red Hat

SUSE Linux

SUSE

Debian

debian

Amazon Machine Image (AMI)

Ubuntu Server 26.04 LTS (HVM), SSD Volume Type

ami-07a00cf47dbbc844c (64-bit (x86)) / ami-0af878be293432b08 (64-bit (Arm))

Virtualization: hvm ENA enabled: true Root device type: ebs

Description

Ubuntu Server 26.04 LTS (HVM),EBS General Purpose (SSD) Volume Type. Support available from Canonical (<http://www.ubuntu.com/>)

Canonical, Ubuntu, 26.04, amd64 resolute image

ArchitectureAMI IDPublish DateUsername

64-bit (x86)

ami-07a00cf47dbbc844c

2026-04-21

ubuntu

Verified provider

▼ Instance type [Info](#) | [Get advice](#)

Instance type

t3.micro

Family: t3 2 vCPU 1 GiB Memory Current generation: true On-Demand Linux base pricing: 0.0112 USD per Hour On-Demand SUSE base pricing: 0.0112 USD per Hour On-Demand Windows base pricing: 0.0204 USD per Hour On-Demand Ubuntu Pro base pricing: 0.0147 USD per Hour On-Demand RHEL base pricing: 0.04 USD per Hour

Free tier eligible

All general purpose

[Compare instance types](#)

Additional costs apply for AMIs with pre-installed software

▼ Key pair (login) [Info](#)

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - required

mydockerkey

[Create new key pair](#)

▼ Network settings [Info](#)

Network [Info](#)

vpc-0538c231a226fe3c9

Subnet [Info](#)

No preference (Default subnet in any availability zone)

Auto-assign public IP [Info](#)

Enable

Firewall (security groups) [Info](#)

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

Create security group

Select existing security group

Common security groups [Info](#)

Select security groups

default sg-06d80e515c32b9515

VPC: vpc-0538c231a226fe3c9

[Compare security groups](#)

Security groups that you add or remove here will be added to or removed from all your network interfaces.

Step no:2 Once the slave instance is launched → Go to Jenkins download for ubuntu → Only install java. No need to install Jenkins.

The slave machine only run the jobs scheduled by the master machine. Hence, it doesn't need Jenkins to be installed.

sudo apt update

sudo apt install fontconfig openjdk-21-jre

java -version

```
ubuntu@ip-172-31-35-205:~$ sudo apt update
sudo apt install fontconfig openjdk-21-jre
java -version
Get:1 http://ap-south-1.ec2.archive.ubuntu.com
Get:2 http://ap-south-1.ec2.archive.ubuntu.com
```

i-0b42305ae83a36aee (Slave instance)

PublicIPs: 35.154.233.206 PrivateIPs: 172.31.35.205

Step no:3 Go to Jenkins Master instance

Generate key using: **ssh-keygen**

Master machine

```
root@ip-172-31-10-184:/home/ubuntu# ssh-keygen
Generating public/private ed25519 key pair.
Enter file in which to save the key (/root/.ssh/id_ed25519):
Enter passphrase for "/root/.ssh/id_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/id_ed25519
Your public key has been saved in /root/.ssh/id_ed25519.pub
```

List the files and hidden folders: **ls-la**

Get inside of .ssh folder: **cd .ssh**

```
ubuntu@ip-172-31-9-29:~$ ls -la
total 28
drwxr-x--- 4 ubuntu ubuntu 4096 Jun  6 10:44 .
drwxr-xr-x 3 root    root   4096 Jun  6 10:42 ..
-rw-r--r-- 1 ubuntu ubuntu  220 Feb 13 12:16 .bash_logout
-rw-r--r-- 1 ubuntu ubuntu 3771 Feb 13 12:16 .bashrc
drwx----- 2 ubuntu ubuntu 4096 Jun  6 10:44 .cache
-rw-r--r-- 1 ubuntu ubuntu  807 Feb 13 12:16 .profile
drwx----- 2 ubuntu ubuntu 4096 Jun  6 10:49 .ssh
ubuntu@ip-172-31-9-29:~$ cd .ssh
ubuntu@ip-172-31-9-29:~/.ssh$ ls -la
total 20
drwx----- 2 ubuntu ubuntu 4096 Jun  6 10:49 .
drwxr-x--- 4 ubuntu ubuntu 4096 Jun  6 10:44 ..
-rw----- 1 ubuntu ubuntu  393 Jun  6 10:42 authorized_keys
-rw----- 1 ubuntu ubuntu  411 Jun  6 10:49 id_ed25519
-rw-r--r-- 1 ubuntu ubuntu  103 Jun  6 10:49 id_ed25519.pub
```

We can see our public key. See the contents using: **cat pubkeyid**

Copy the contents

```
ubuntu@ip-172-31-9-29:~/.ssh$ cat id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIAi6q3NNmY2KjWKmxCUbuMFUCz0sgUtj+iIT
ubuntu@ip-172-31-9-29:~/.ssh$
```

i-04893031bb2ff442c (Jenkins-instance)

PublicIPs: 13.233.103.232 PrivateIPs: 172.31.9.29

Step no:4 Now go the slave machine

List the file and hidden folders: **ls -la**

Open .ssh: **cd .ssh**

We can see the authorised key file inside it.

```
root@ip-172-31-35-205:/home# cd /home/ubuntu
root@ip-172-31-35-205:/home/ubuntu# ls -la
total 28
drwxr-x--- 4 ubuntu ubuntu 4096 Jun  6 10:47 .
drwxr-xr-x 3 root  root  4096 Jun  6 10:43 ..
-rw-r--r-- 1 ubuntu ubuntu  220 Feb 13 12:16 .bash_logout
-rw-r--r-- 1 ubuntu ubuntu 3771 Feb 13 12:16 .bashrc
drwx----- 2 ubuntu ubuntu 4096 Jun  6 10:47 .cache
-rw-r--r-- 1 ubuntu ubuntu  807 Feb 13 12:16 .profile
drwx----- 2 ubuntu ubuntu 4096 Jun  6 10:43 .ssh
root@ip-172-31-35-205:/home/ubuntu# cd .ssh
root@ip-172-31-35-205:/home/ubuntu/.ssh# ls -la
total 12
drwx----- 2 ubuntu ubuntu 4096 Jun  6 10:43 .
drwxr-x--- 4 ubuntu ubuntu 4096 Jun  6 10:47 ..
-rw----- 1 ubuntu ubuntu  393 Jun  6 10:43 authorized_keys
```

Open the authorised keys file using the text editor command: **nano authorised_keys**

Paste the content copied from the public key present in the Jenkins master machine without deleting the existing contents.

```
root@ip-172-31-35-205:/home/ubuntu/.ssh# nano authorized_keys
root@ip-172-31-35-205:/home/ubuntu/.ssh# cat authorized_keys
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQGCzW3upB5iy4w7Mu/NsPRYURB
NWduQALZN1PKqIzGZbYbQe2IC7o2sxmS2WCXcBPIkfinMRiP20ZIaVxiHYowTs
f3jj9tcj mydockerkey
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIAi6q3NNmY2KjWKmxCUbuMFUCz
root@ip-172-31-35-205:/home/ubuntu/.ssh#
```

i-0b42305ae83a36aee (Slave instance)

PublicIPs: 35.154.233.206 PrivateIPs: 172.31.35.205

Step no:5 Open the Jenkins page → Choose **Build Executor Status**

 **Jenkins**

[+ New Item](#)
[Build History](#)
[Project Relationship](#)
[Check File Fingerprint](#)

Build Queue

No builds in the queue.

Build Executor Status

0/2

Click **new node**

enkins / Nodes

Nodes

[+ New Node](#)[Configure](#)

Add, remove, control and monitor the various nodes that Jenkins runs jobs on.

S	Name ↓	Architecture	Clock Difference	Free Disk Space	Free Swap Space	Free Temp Space	Resp
	Built-In Node	Linux (amd64)	In sync	22.28 GiB	 0 B	3.79 GiB	
	Data obtained	17 min	17 min	17 min	17 min	17 min	

Name the node as: **new**

Choose **Permanent agent** → create.

New node

Node name

Type

☒ Permanent Agent

Adds a plain, permanent agent to Jenkins. This is called "permanent" because Jenkins doesn't provide higher level of integration with these agents, such as dynamic provisioning. Select this type if no other agent types apply — for example such as when you are adding a physical computer, virtual machines managed outside Jenkins, etc.

[Create](#)

Step no:6 Go to the slave console and create a directory called Jenkins under the **/home/ubuntu**

```
root@ip-172-31-32-236:/home/ubuntu/.ssh# cd ..
root@ip-172-31-32-236:/home/ubuntu# pwd
/home/ubuntu
root@ip-172-31-32-236:/home/ubuntu# mkdir jenkins
root@ip-172-31-32-236:/home/ubuntu# cd jenkins
root@ip-172-31-32-236:/home/ubuntu/jenkins# pwd
/home/ubuntu/jenkins
```

In the Jenkins page, provide the **absolute path in remote root directory**.

Labels: **mynew** (this is what we use in the Jenkinsfile).

Name ?

new

Description ?

Plain text [Preview](#)

Number of executors ?

1

Remote root directory ?

/home/ubuntu/jenkins

Labels ?

mynew

Host: Paste **the public ip of the slave machine**

Launch method: **Via SSH**

Labels ?

mynew

Usage ?

Use this node as much as possible

Launch method ?

Launch agents via SSH

Host ?

35.154.233.206

Credentials ?

- none -

 The selected credentials cannot be found

Host Key Verification Strategy ?

Known hosts file Verification Strategy

Credentials: **Add → SSH Username with private key → next.**

Here we will use our key provided in the instance creation.

Add Credentials

×

Select a type of credential

 Username with password
Commonly used for authentication to services like Git, APIs, or registries.

 GitHub App

 SSH Username with private key

 Secret file

 Secret text

 Certificate
Upload a PKCS#12 or PEM encoded certificate and private key.

Next

Id: **ubuntu**

Username: **ubuntu** (username of our instance (slave))

Choose: **Enter key directly**

Open the downloaded keypair file in our local, copy the contents and paste it there.

< Add SSH Username with private key

×

ID ?

ubuntu

Description ?

Username

ubuntu

☐ Treat username as secret ?

Private Key

☒ Enter directly

Key

Enter New Secret Below

Create

Step no:7 Now it will popup in the credentials choose it and Ensure Manually trusted key verification strategy.

The availability of agent must be online as much as possible.

Save and apply.

Launch method ?

Launch agents via SSH

Host ?

35.154.233.206

Credentials ?

ubuntu

Host Key Verification Strategy ?

Manually trusted key Verification Strategy

☐ Require manual verification of initial connection ?

Advanced ▾

Availability ?

Keep this agent online as much as possible

Error : sometimes the agent will on work and doesn't come to online.

Follow the below to make the agent back to online.

* Fix the Ownership

The most likely culprit is that the **/home/ubuntu/jenkins** folder was accidentally created by the root user, so the ubuntu user can't write to it. Grant ownership back to the ubuntu user:

sudo chown ubuntu:ubuntu /home/ubuntu/Jenkins

* Fix the Permissions

Ensure that the ubuntu user has full read, write, and execute permissions on that directory:

chmod 755 /home/ubuntu/Jenkins

The agent will come to alive after refresh.

Nodes

+ New Node

Configure Monitors



Add, remove, control and monitor the various nodes that Jenkins runs jobs on.

S	Name ↓	Architecture	Clock Difference	Free Disk Space	Free Swap Space	Free Temp Space	Response Time
	Built-In Node	Linux (amd64)	In sync	24.88 GiB	0 B	1.86 GiB	0ms
	new	Linux (amd64)	In sync	3.90 GiB	0 B	952.45 MiB	5ms
Data obtained		1 min 7 sec	1 min 7 sec	1 min 7 sec	1 min 7 sec	1 min 7 sec	1 min 7 sec

Icon: S M L

Legend

Step no:8 Now open the Jenkins and create a pipeline, to execute via our new agent.

New Item

Enter an item name

My-test-agent

Select an item type



Pipeline

Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.

The Pipeline Code (Jenkinsfile)

To execute a build job on this specific agent, use a Jenkinsfile. You must restrict the job to this node by targeting its **Name** or **Label** (which you set as new in your configuration).

Agent name: **new** , Label: **mynew**

Definition

Pipeline script

Script ?

```
1 pipeline {
2   agent {
3     // Targets the agent named "new"
4     node {
5       label 'mynew'
6     }
7   }
8   stages {
9     stage('Hello World') {
10      steps {
11        // This command will execute directly on your ubuntu agent
12        sh 'echo "Running on the remote ubuntu agent!"'
13        sh 'df -h' // Checks disk space during the build
14      }
15    }
16  }
```

Save and apply.

Step no:9 Create a build for our new item.

Once the build is successful open the console output.

The screenshot shows the Jenkins web interface. At the top, it says 'Jenkins / My-test-agent'. Below this, there's a 'Status' tab and a green checkmark icon next to 'My-test-agent'. A dropdown menu is open, showing various actions: Changes, Console Output, Edit Build Information, Delete build '#1', Timings, Pipeline Overview, Restart from Stage, Replay, Pipeline Steps, and Workspaces. On the left sidebar, there are links for Changes, Build Now, Configure, Delete Pipeline, Stages, Rename, and Pipeline Syntax. Below the sidebar, there's a 'Builds' section with a filter input and a list of builds. The first build is '#1' at '11:32 AM' with a green checkmark icon.

The agent will be now show as busy

The screenshot shows the Jenkins web interface. At the top, it says 'Check file fingerprint'. Below this, there's a 'Build Queue' section with a dropdown arrow and the text 'No builds in the queue.' To the right, there's a table showing the status of the 'My-test-agent' pipeline. The table has columns for 'S', 'W', 'Name', and 'Last Success'. The 'My-test-agent' pipeline is shown with a green checkmark icon, a sun icon, and a duration of '3 min 21 sec'. Below the table, there's a 'Build Executor Status' section with a dropdown arrow and the text 'built-in node + 1 agent (0 of 1 executors busy)'. To the right of this section, there's a row of icons: 'S', 'M', and 'L'.

We can see the execution is successful and a workspace has been created for this execution in the slave machine. Copy the path **/home/ubuntu/Jenkins/workspace**.

Console

```
Started by user Nithya Sri
[Pipeline] Start of Pipeline
[Pipeline] node
Running on new in /home/ubuntu/jenkins/workspace/My-test-agent
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Hello World)
[Pipeline] sh
+ echo Running on the remote ubuntu agent!
Running on the remote ubuntu agent!
[Pipeline] sh
```

```
none          1.0M      0  1.0M   0% /run/credentials/getty@tty1.service
none          1.0M      0  1.0M   0% /run/credentials/serial-getty@ttyS0.service
tmpfs         191M   8.0K  191M   1% /run/user/1000
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Step 10: Open the path in the slave machine we can see the My-test-agent present inside it.

```
ubuntu@ip-172-31-35-205:/$ cd /home/ubuntu/jenkins
ubuntu@ip-172-31-35-205:~/jenkins$ ls
caches  remoting  remoting.jar  workspace
ubuntu@ip-172-31-35-205:~/jenkins$ cd workspace
ubuntu@ip-172-31-35-205:~/jenkins/workspace$ ls
My-test-agent  My-test-agent@tmp
ubuntu@ip-172-31-35-205:~/jenkins/workspace$
```

i-0b42305ae83a36aee (Slave instance)

PublicIPs: 35.154.233.206 PrivateIPs: 172.31.35.205